

# Natural Language Understanding Assignment 01

## Problem 04: SPORTS OR POLITICS

Sumeet S Patil (B22CS052)

### 1 Introduction

Text classification is a classical Natural Language Processing (NLP) task, with applications in spam filtering, content recommendation system, news categorization, information retrieval and many more. Automatically categorizing news articles into specific domains helps for efficient content organization and retrieval at a large scale.

In this project, we are doing binary text classification problem of categorizing news articles into two domains: **Sports** and **Politics**. The objective is to compare multiple classical machine learning techniques and feature representations to determine the best combination for this task.

We experimented with three different feature extractions - **Bag of Words (BoW)**, **n-grams** and **TF-IDF**, while evaluating three machine learning models: **Multinomial Naive Bayes**, **Logistic Regression** and **Linear Support Vector Machine**.

**Github page for this problem:** [Repository](#)

### 2 Dataset Description

The dataset consists of 1434 news articles, collected from a publicly available Github repository Dataset source. Each article is a medium-length paragraph containing domain-specific content.

#### 2.1 Dataset extraction from Repository

The repository houses multiple news articles related to many genres and the articles are classified into positive and negative sub categories, each into a compressed folder. Each compressed folder consists of 1000 article json files.

To keep things simple for this assignment, we used 1 positive sports, 1 positive politics, 1 negative sports and 1 negative politics article group (of 1000). From these downloaded article group, we retained 500 from each group and discarded the rest, leaving us with 2000 articles, 1000 for each category. Then, the retained 2000 articles were preprocessed to build the dataset needed for this task.

| Class    | Number of Records | Percentage | Average Article Length |
|----------|-------------------|------------|------------------------|
| Sports   | 866               | 60%        | 3511                   |
| Politics | 568               | 40%        | 3367                   |

Table 1: Dataset Distribution

## 2.2 Preprocessing Dataset

The raw dataset obtained from the repository was preprocessed to build a suitable dataset format for this task. The steps taken were as follows:

- **Raw article text extraction:** Each article json had a key-value pair named **text**, which contained the article content. So, that value was extracted as the article paragraph.
- **Label extraction:** Label of the article was extracted from the **categories** key-value pair, and sports/politics substring matching was done with each category label in the list to find the right label.
- **Text processing:** We removed boilerplate text (copyright, author lines, metadata), so that these unrelated words don't hinder the dataset and in-turn the model. All the characters were lower-cases and excessive whitespace was removed.
- **Short documents elimination:** After the text processing, the articles with less than 50 words were discarded. This decision was taken to reduce the noise in the dataset due to short documents, which contributed less to the vocabulary and more to noise.

After all the above steps, we saved the processed dataset in comma separated format for better usability in later stages of the problem. After pre-processing, we were left with 1434 document articles. All the pre-processing steps are done by the notebook named **preprocess\_dataset.ipynb** in the repository **Repository**

## 2.3 Dataset statistics

Even though raw data has equal contribution from sports and political articles, the processed dataset showed slight class imbalance as shown in Table 01. The average article lengths are quite close, while the class imbalance might lead to misclassifications of political articles.

## 2.4 Dataset split

The dataset was split into train, validation and test dataset with ration 80:10:10 with stratified sampling. This was done to account for the class imbalance in the dataset.

## 3 Feature Representation

We have used three features for this assignment, Unigram, Bag of Words and TF-IDF features.

### 3.1 Unigram features

In this feature type, every document is represented as a vector of word frequencies. No, local context was captured in this representation. These features indicated the category-specific vocabulary. Higher order n-grams weren't used due to their sparsity and diminishing returns in performance.

### 3.2 Bag of Words (Bigram-Unigram)

This feature type uses both unigram and bigram representations of the document. Thus, this feature type is now able to account for some local context and compound words like prime minister or world cup are now recognized as category-specific vocabulary. However, still we do not capture long range dependencies.

### 3.3 TF-IDF

Term Frequency-Inverse Document Frequency weights each term as per its frequency in a document and its presence across multiple documents. This representation reduces the impact of commonly occurring words and emphasizes discriminative terms.

## 4 Machine Learning Models

This section describes the machine learning models used for this assignment.

### 4.1 Multinomial Naive Bayes

Naive Bayes, being a very simple generative model, was chosen to set up a strong baseline for this task. It was chosen for its simplicity and computational efficiency.

### 4.2 Logistic Regression

Logistic Regression, being a simple discriminative model, was chosen again for its simplicity and efficiency in computation. It performs well in sparse high-dimensional spaces.

### 4.3 Linear Support Vector Machine

Linear SVM finds a hyperplane that maximizes the margin between classes. It is particularly effective for text classification due to its robustness in high-dimensional feature spaces.

## 5 Training and Testing

A pipeline was set up post the train test and validation split that extracted features, trained the specific model, validated the validation set, tested on the test data and all statistics needed were returned, with confusion matrix plotted for each combination.

Basic Machine Learning libraries, which includes sklearn, pandas, numpy were used for mathematical operations and machine learning models, while seaborn and matplotlib were used for plotting confusion matrix.

Thus, summary of the setup is as follows:

- All models used from sklearn library
- Maximum feature size: 10000
- English stop words removed
- Stratified sampling done (to counter class imbalance).
- Evaluation Metrics:
  - Accuracy
  - Precision
  - Recall
  - F1-score
  - Visualization using confusion matrix

## 6 Results

After training and testing the models, the results were obtained as discussed below. We shall discuss results model wise.

### 6.1 Multinomial Naive Bayes

#### 6.1.1 Feature Sensitivity

This model performed best with Unigram features (Accuracy of 0.9167) compared to TF-IDF (0.9097). This often happens because of the independence assumption. Simple frequency counts in unigram outperform other schemes when the vocabulary is distinct enough between the two classes, which has happened in this case.

#### 6.1.2 Confusion Matrices Analysis

The confusion matrices show that with TF-IDF, the model became highly specialized sports class, but it started to misclassify politics records. This is an direct consequence of the original class bias in the dataset, which in turn is reflected the tf-idf features of the words.

| Feature Type | Val Accuracy | Accuracy | Precision | Recall | F1-Score |
|--------------|--------------|----------|-----------|--------|----------|
| Unigram      | 0.8881       | 0.9167   | 0.9175    | 0.9167 | 0.9169   |
| Bag of Words | 0.9027       | 0.9027   | 0.9037    | 0.9028 | 0.9030   |
| TF-IDF       | 0.9097       | 0.9097   | 0.9143    | 0.0907 | 0.9080   |

Table 2: Multinomial Naive Bayes Results

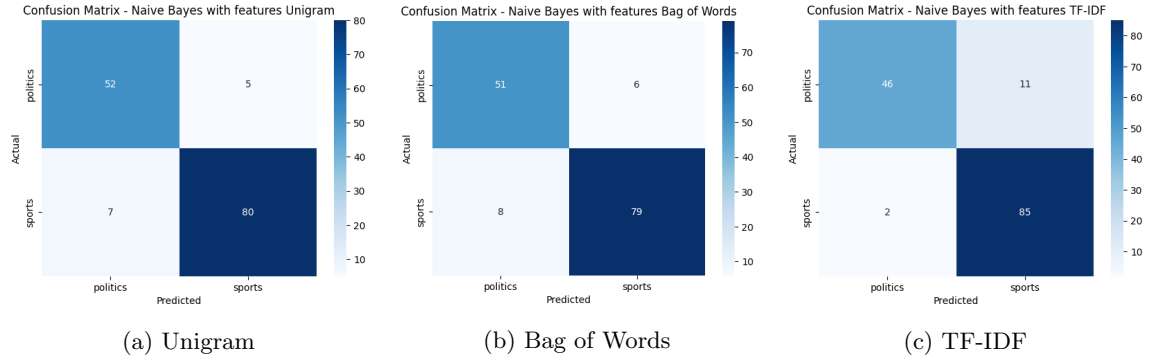


Figure 1: Confusion Matrices with Multinomial Naive Bayes

## 6.2 Logistic Regression

### 6.2.1 Feature Sensitivity

This model showed consistent and stable performance across all features ( 0.90 to 0.91 accuracy). As a discriminative model, it excelled at finding a clear boundary regardless of the features and class bias.

### 6.2.2 Confusion Matrices Analysis

Using TF-IDF allowed this model to reach its highest precision, with perfect recall for sports, while some misclassifications of politics records, again a consequence of class bias.

| Feature Type | Val Accuracy | Accuracy | Precision | Recall | F1-Score |
|--------------|--------------|----------|-----------|--------|----------|
| Unigram      | 0.9510       | 0.9028   | 0.9028    | 0.9028 | 0.9028   |
| Bag of Words | 0.9371       | 0.9097   | 0.9101    | 0.9097 | 0.9099   |
| TF-IDF       | 0.9161       | 0.9097   | 0.9215    | 0.9097 | 0.9071   |

Table 3: Logistic Regression Results

## 6.3 Linear SVM

### 6.3.1 Feature Sensitivity

This model shows the actual gain by using TF-IDF rather than Unigram, where the improvement on accuracy went from 0.87 in Unigram to nearly 0.92 in TF-IDF. the normalized and scaled values of TF-IDF helped the model find an effective hyper plane to differentiate between the two categories.

### 6.3.2 Confusion Matrices Analysis

TF-IDF confusion matrix is a promising one with balanced error rate, thus proving that TF-IDF has reduced the noise for this model.

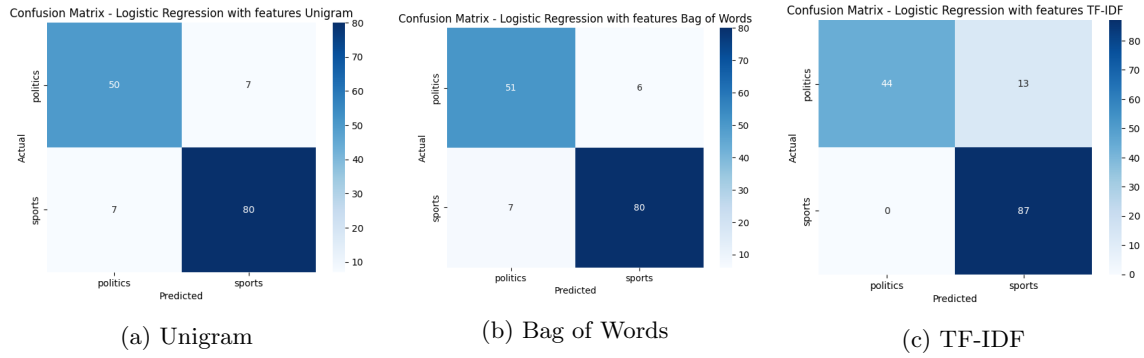


Figure 2: Confusion Matrices with Logistic Regression

## 6.4 Overall Analysis

Linear SVM with TF-IDF is the strongest configuration achieved in this exercise. Naturally, in text classification, rare words carry more discriminative power than frequent ones. TF-IDF rightly captures this nature and helps reach a better solution. As far as validation results as concerned, even though logistic regression shows higher validation accuracies, but they underperformed on test data, suggesting, they might have slightly over-fitted on the training data. Linear SVM on the other side shows better alignment between validation and test accuracies, suggesting the generalization of learning.

| Feature Type | Val Accuracy | Accuracy | Precision | Recall | F1-Score |
|--------------|--------------|----------|-----------|--------|----------|
| Unigram      | 0.9091       | 0.8681   | 0.8677    | 0.8681 | 0.8678   |
| Bag of Words | 0.9021       | 0.8750   | 0.8750    | 0.8750 | 0.8750   |
| TF-IDF       | 0.9301       | 0.9236   | 0.9237    | 0.9236 | 0.9232   |

Table 4: Linear SVM Results

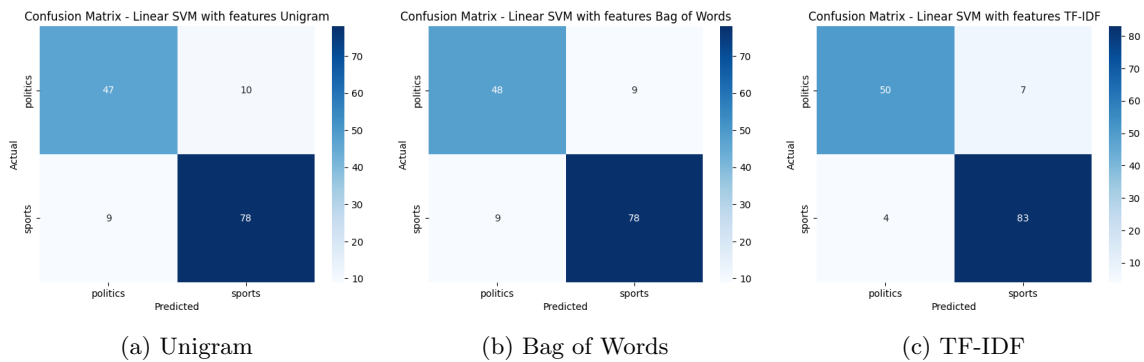


Figure 3: Confusion Matrices with Linear SVM

## 7 Limitations

This setup has a few limitations, which we will enumerate and discuss here.

- Class Bias: Despite being low (60-40), class bias becomes a major factor in the training of the model, reflected by the confusion matrices of various models.
- Freshness of dataset: The dataset is relatively old (2024-2025), thus these models may under perform on the new document records, mainly because of the dynamic nature of politics and sports fields, thus with multiple unknown words or confusing words getting added in the newer document records.
- Size of dataset: The dataset is quite small, so we solely rely on discriminative models (reflected by their performance), rather than truly understanding the semantics of those articles.
- We don't do any deep semantic modeling here, just classical ML techniques with very basic features are tested.

## 8 Conclusion

This study evaluated classical machine learning models for domain classification of news articles into Sports and Politics categories. Key findings were that Linear SVM with TF-IDF resulted in the best performance, highlighting the strength of both the model and feature. We could also see that a complex task as text classification also could be modeled sufficiently accurate with classical machine learning techniques.